



Trabajo Final Diseño Software

Sistema de reparto de comida - Uber Eats

Daniel Lopez - Jose Donoso - David San Martin

Rodrigo Castro - Jacob Gomez

ASIGNATURA: DISEÑO SOFTWARE

GRUPO: A

GRADO: MAIIS Y FIIS

CURSO ACADÉMICO: 2º

1. Introducción

El proyecto desarrollado consiste en una simulación por consola de una plataforma de reparto de comida a domicilio inspirada en aplicaciones como Uber Eats. La idea principal del sistema es representar el proceso completo de creación y gestión de un pedido, desde que un cliente selecciona un restaurante y varios productos del menú, hasta que el pedido pasa por diferentes estados y finalmente es entregado o cancelado.

El objetivo principal no era crear una aplicación con interfaz gráfica, sino aplicar correctamente varios patrones de diseño vistos durante la asignatura de Diseño de Software. Por ello, el proyecto se ha centrado en que la estructura del código sea clara, mantenible y que cada patrón tenga una función concreta dentro del caso de uso.

La aplicación permite registrar usuarios de distintos tipos, crear pedidos, personalizar productos, seleccionar métodos de pago, cambiar el estado de los pedidos, notificar a los usuarios implicados y gestionar los pedidos activos o finalizados. Todo el funcionamiento se realiza mediante consola.

2. Descripción general del caso de uso

El caso de uso elegido es una plataforma de comida a domicilio. En este sistema intervienen principalmente tres tipos de usuarios:

- Cliente.
- Restaurante.
- Repartidor.

El cliente realiza un pedido a un restaurante. El pedido contiene uno o varios productos del menú, que pueden personalizarse mediante extras, cambios de tamaño, eliminación de ingredientes o descuentos. Una vez creado el pedido, se le asigna un repartidor disponible y se selecciona un método de pago.

Después, el pedido va avanzando por un ciclo de vida compuesto por varios estados:

1. Pendiente.
2. En preparación.
3. En reparto.
4. Entregado.

Además, el pedido puede cancelarse siempre que no haya sido entregado todavía.

El sistema también notifica automáticamente a las partes implicadas cuando cambia el estado de un pedido. De esta forma, el cliente, el restaurante y el repartidor reciben información sobre la evolución del pedido.

3. Contexto, restricciones y alcance

3.1. Contexto

El proyecto se ha desarrollado como práctica final de la asignatura de Diseño de Software. Se ha optado por un caso de uso libre, buscando una situación realista donde tuviera sentido combinar varios patrones de diseño.

La idea de una plataforma tipo Uber Eats resulta adecuada porque permite representar varios problemas habituales en el diseño de software:

- Creación de distintos tipos de usuarios.
- Gestión centralizada de datos del sistema.
- Cambio de estado de los pedidos.
- Notificación automática a diferentes usuarios.
- Selección dinámica de un método de pago.
- Personalización flexible de productos.
- Simplificación de la creación de pedidos complejos.

3.2. Restricciones

Las principales restricciones del proyecto son:

- El proyecto está desarrollado únicamente en Java.
- La aplicación funciona por consola.
- No se utiliza una base de datos real.
- Los datos se almacenan en listas en memoria.
- No se implementa una interfaz gráfica.

- El objetivo principal es demostrar el uso correcto de patrones de diseño.

Al no usar base de datos, la clase **ServerManager** actúa como una base de datos simulada, almacenando listas de clientes, restaurantes, repartidores y pedidos.

3.3. Alcance

El alcance del proyecto incluye:

- Registro de clientes, restaurantes y repartidores.
- Eliminación de usuarios registrados.
- Creación de pedidos.
- Asignación automática de repartidor.
- Personalización de productos del menú.
- Selección de método de pago.
- Cambio de estado de los pedidos.
- Cancelación de pedidos.
- Notificación a cliente, restaurante y repartidor.
- Visualización de pedidos activos.
- Visualización y eliminación de pedidos finalizados o cancelados.
- Ejecución de pruebas automáticas para comprobar los patrones principales.

No se han incluido funcionalidades más avanzadas como login, persistencia en base de datos, gestión real de direcciones, control de stock, interfaz gráfica o conexión con servicios externos de pago.

4. Estructura general del proyecto

El proyecto está organizado dentro del paquete `com.ubereats`, separando las clases principales y los patrones de diseño en distintos paquetes.

La estructura general del sistema puede observarse en la Figura ??, donde se muestran las clases principales, sus relaciones y los patrones de diseño implementados.

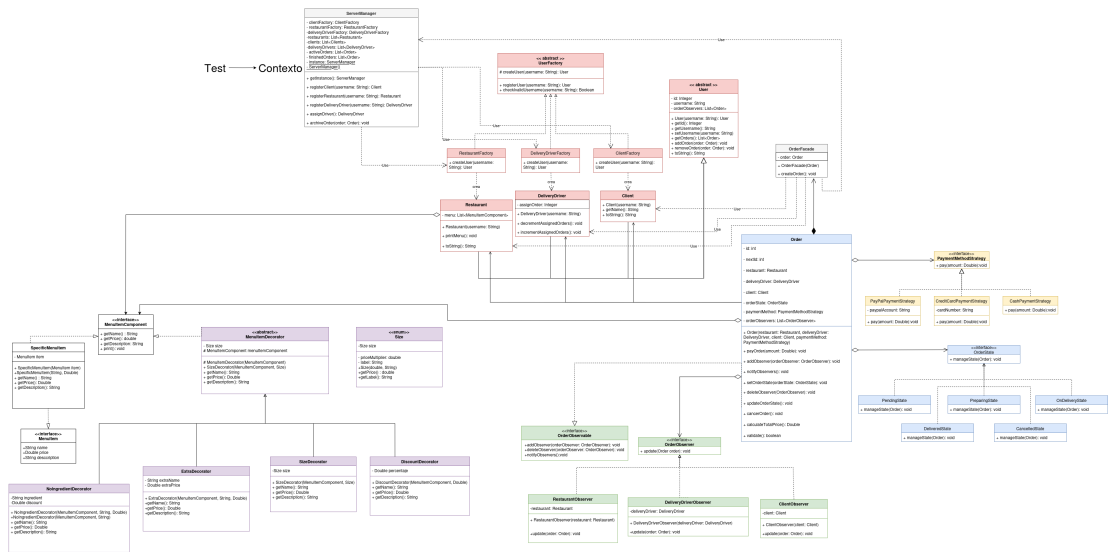


Figura 1: Diagrama UML del sistema

Esta organización permite localizar fácilmente qué clases pertenecen a cada patrón. Además, separa las entidades principales del dominio, como **Client**, **Restaurant**, **DeliveryDriver**, **Order** y **User**, de las clases que implementan los patrones.

5. Componentes principales del sistema

5.1. User

User es una clase abstracta que representa a cualquier usuario del sistema. De ella heredan los tres tipos principales de usuario:

- **Client**
- **Restaurant**
- **DeliveryDriver**

La clase contiene atributos comunes como el identificador, el nombre de usuario y una lista de pedidos asociados. Esto permite que cada usuario pueda almacenar sus pedidos, aunque cada tipo de usuario los use con un sentido diferente.

Por ejemplo, el cliente almacena los pedidos que ha realizado, el restaurante almacena los pedidos que debe preparar y el repartidor almacena los pedidos que tiene asignados.

5.2. Client

`Client` representa al cliente que realiza pedidos. Es una clase sencilla que hereda de `User` y permite identificar al cliente dentro del sistema.

5.3. Restaurant

`Restaurant` representa al restaurante que recibe pedidos. Además de heredar de `User`, contiene una lista de productos del menú. Estos productos se almacenan como `MenuItemComponent`, lo que permite que puedan ser productos básicos o productos decorados con extras, tamaños, descuentos o ingredientes eliminados.

5.4. DeliveryDriver

`DeliveryDriver` representa al repartidor. Además de los datos comunes heredados de `User`, contiene un contador de pedidos asignados. Este contador se utiliza para asignar nuevos pedidos al repartidor con menor carga de trabajo.

5.5. Order

`Order` representa un pedido. Es una de las clases más importantes del proyecto porque conecta varios patrones a la vez.

Un pedido contiene:

- Cliente.
- Restaurante.
- Repartidor.
- Lista de productos.
- Estado actual.
- Método de pago.
- Lista de observadores.

Además, `Order` implementa la interfaz `OrderObservable`, por lo que puede añadir observadores, eliminarlos y notificarles cuando cambia el estado del pedido.

5.6. **ServerManager**

ServerManager actúa como gestor central de la aplicación. Contiene las listas de clientes, restaurantes, repartidores y pedidos. Además, se encarga de registrar usuarios utilizando las factorías correspondientes y de asignar un repartidor a cada pedido.

Esta clase se ha implementado como Singleton para garantizar que exista una única instancia central del servidor durante toda la ejecución.

5.7. **OptionsMenu**

OptionsMenu contiene la lógica del menú por consola. Desde esta clase se pueden registrar usuarios, eliminar usuarios, simular pedidos, actualizar estados, cancelar pedidos y consultar pedidos activos o finalizados.

Esta clase conecta la interacción del usuario con el resto del sistema.

5.8. **Test y TestAutomatico**

Test contiene una ejecución interactiva por consola, donde el usuario puede seleccionar opciones manualmente.

TestAutomatico contiene pruebas automáticas para comprobar el funcionamiento de los patrones principales. Esta clase es útil para verificar rápidamente que el sistema funciona correctamente sin tener que recorrer manualmente todo el menú.

6. **Patrones de diseño implementados**

6.1. **Singleton**

Problema detectado

El sistema necesitaba una clase que gestionara de forma centralizada los datos principales de la aplicación: clientes, restaurantes, repartidores y pedidos. Si se creasen varias instancias de esta clase, cada una tendría sus propias listas y se perdería la coherencia de los datos.

Por ejemplo, un pedido creado en una instancia podría no aparecer en otra instancia diferente.

Solución aplicada

Se ha aplicado el patrón Singleton en la clase **ServerManager**. Esta clase tiene una instancia estática privada y un constructor privado. La única forma de acceder a ella es mediante el método `getInstance()`.

De esta manera, todo el programa utiliza la misma instancia del gestor central.

Clases participantes

- **ServerManager**: clase Singleton.
- **Test**, **OptionsMenu**, **OrderFacade** y **TestAutomatico**: clases cliente que acceden a la instancia mediante `getInstance()`.

Justificación

Este patrón encaja bien en el proyecto porque **ServerManager** representa el servidor central de la aplicación. Al ser una simulación sin base de datos real, esta clase funciona como punto único de almacenamiento y gestión.

6.2. Factory Method

Problema detectado

El sistema necesitaba crear distintos tipos de usuario: cliente, restaurante y repartidor. Aunque todos heredan de **User**, cada uno se instancia con una clase concreta diferente.

Si la lógica de creación estuviera mezclada directamente en el menú o en el servidor, el código sería menos flexible y más difícil de ampliar.

Solución aplicada

Se ha aplicado el patrón Factory Method mediante una clase abstracta **UserFactory** y varias factorías concretas:

- **ClientFactory**
- **RestaurantFactory**
- **DeliveryDriverFactory**

La clase `UserFactory` define el método general para registrar un usuario y delega la creación concreta en el método protegido `createUser`.

Cada factoría concreta implementa ese método devolviendo el tipo de usuario correspondiente.

Clases participantes

- `UserFactory`: creador abstracto.
- `ClientFactory`: creador concreto de clientes.
- `RestaurantFactory`: creador concreto de restaurantes.
- `DeliveryDriverFactory`: creador concreto de repartidores.
- `User`: producto abstracto.
- `Client`, `Restaurant`, `DeliveryDriver`: productos concretos.
- `ServerManager`: clase que utiliza las factorías para registrar usuarios.

Justificación

Este patrón permite separar la lógica de creación de usuarios del resto de la aplicación. Además, facilita añadir nuevos tipos de usuario en el futuro sin modificar el código existente.

6.3. Observer

Problema detectado

Cuando un pedido cambia de estado, varios objetos necesitan enterarse del cambio:

- El cliente debe saber cómo avanza su pedido.
- El restaurante debe saber cuándo el pedido pasa a reparto o se cancela.
- El repartidor debe saber cuándo un pedido se entrega o se cancela.

Si la clase `Order` llamara directamente a métodos concretos de cada clase, quedaría muy acoplada a `Client`, `Restaurant` y `DeliveryDriver`.

Solución aplicada

Se ha aplicado el patrón Observer. La clase `Order` actúa como objeto observable, mientras que los observadores concretos son:

- `ClientObserver`
- `RestaurantObserver`
- `DeliveryDriverObserver`

La interfaz `OrderObserver` define el método `update(Order order)`, que ejecuta cada observador cuando recibe una notificación.

La interfaz `OrderObservable` define los métodos:

- `addObserver`
- `deleteObserver`
- `notifyObservers`

Cuando cambia el estado del pedido, se llama a `notifyObservers()` para avisar a todos los observadores registrados.

Clases participantes

- `OrderObservable`: interfaz del observable.
- `Order`: sujeto observable.
- `OrderObserver`: interfaz de los observadores.
- `ClientObserver`: observador del cliente.
- `RestaurantObserver`: observador del restaurante.
- `DeliveryDriverObserver`: observador del repartidor.

Justificación

Este patrón permite que el pedido notifique cambios sin conocer en detalle cómo debe reaccionar cada usuario. Cada observador contiene su propia lógica.

Por ejemplo, el repartidor elimina de su lista un pedido cuando está entregado o cancelado, mientras que el restaurante lo elimina cuando pasa a reparto o se cancela.

6.4. State

Problema detectado

Un pedido cambia de comportamiento según su estado. No es lo mismo un pedido pendiente que uno en preparación, en reparto, entregado o cancelado.

Una solución sencilla habría sido utilizar muchos `if` o `switch` dentro de la clase `Order`, pero eso habría hecho que la clase creciera demasiado y fuese más difícil de mantener.

Solución aplicada

Se ha aplicado el patrón State. Cada estado del pedido se representa mediante una clase distinta que implementa la interfaz `OrderState`.

Los estados implementados son:

- `PendingState`
- `PreparingState`
- `OnDeliveryState`
- `DeliveredState`
- `CancelledState`

Cada estado sabe cómo gestionar el avance al siguiente estado mediante el método `manageState(Order order)`.

El flujo normal es:

`PendingState` → `PreparingState` → `OnDeliveryState` → `DeliveredState`

Además, el pedido puede pasar a `CancelledState` si se cancela antes de ser entregado.

Clases participantes

- `OrderState`: interfaz de estado.
- `PendingState`: estado pendiente.
- `PreparingState`: estado en preparación.

- `OnDeliveryState`: estado en reparto.
- `DeliveredState`: estado entregado.
- `CancelledState`: estado cancelado.
- `Order`: contexto que contiene el estado actual.

Justificación

Este patrón permite separar la lógica de cada estado en una clase diferente. Así, la clase `Order` no necesita conocer todos los detalles del cambio de estado, sino que delega el comportamiento en el estado actual.

6.5. Strategy

Problema detectado

El sistema permite pagar un pedido usando distintos métodos de pago:

- Efectivo.
- Tarjeta.
- PayPal.

Cada método de pago tiene una forma distinta de ejecutarse. Si toda la lógica estuviera dentro de `Order`, habría que añadir condicionales para distinguir entre los diferentes métodos.

Solución aplicada

Se ha aplicado el patrón Strategy. Se define una interfaz común `PaymentMethodStrategy` con el método `pay(Double amount)`.

Después, cada método de pago se implementa como una estrategia concreta:

- `CashPaymentStrategy`
- `CardPaymentStrategy`
- `PaypalPaymentStrategy`

La clase `Order` solo guarda una referencia a `PaymentMethodStrategy` y llama al método `pay`, sin importar qué estrategia concreta se esté usando.

Clases participantes

- `PaymentMethodStrategy`: interfaz de estrategia.
- `CashPaymentStrategy`: pago en efectivo.
- `CardPaymentStrategy`: pago con tarjeta.
- `PaypalPaymentStrategy`: pago con PayPal.
- `Order`: contexto que utiliza la estrategia.

Justificación

Este patrón permite cambiar el método de pago en tiempo de ejecución. Además, hace que el sistema sea más fácil de ampliar, ya que se podrían añadir nuevos métodos de pago creando nuevas clases que implementen la interfaz.

6.6. Decorator

Problema detectado

Los productos del menú podían necesitar personalizaciones. Por ejemplo:

- Añadir un extra.
- Cambiar el tamaño.
- Quitar un ingrediente.
- Aplicar un descuento.

Crear una clase diferente para cada combinación posible habría sido poco práctico. Por ejemplo, habría que crear clases como `HamburguesaConQueso`, `HamburguesaGrande`, `HamburguesaGrandeConQuesoSinCebolla`, etc.

Solución aplicada

Se ha aplicado el patrón Decorator. Se define una interfaz `MenuItemComponent` que representa cualquier producto que pueda estar en un pedido.

La clase `BasicMenuItem` representa un producto básico. A partir de ahí, los decoradores envuelven ese producto y modifican su nombre, descripción o precio.

Los decoradores implementados son:

- **ExtraDecorator**: añade un extra y suma precio.
- **SizeDecorator**: cambia el tamaño y aplica un multiplicador.
- **NoIngredientDecorator**: elimina un ingrediente y puede aplicar descuento.
- **DiscountDecorator**: aplica un porcentaje de descuento.

También se usa el enum **Size** para representar los tamaños disponibles:

- **SMALL**
- **MEDIUM**
- **LARGE**
- **EXTRA_LARGE**

Clases participantes

- **MenuItemComponent**: componente común.
- **BasicMenuItem**: componente concreto.
- **MenuItemDecorator**: decorador abstracto.
- **ExtraDecorator**: decorador concreto.
- **SizeDecorator**: decorador concreto.
- **NoIngredientDecorator**: decorador concreto.
- **DiscountDecorator**: decorador concreto.
- **Size**: enum de tamaños.

Justificación

Este patrón es adecuado porque permite combinar personalizaciones de manera flexible. Un mismo producto puede tener varios decoradores encadenados sin modificar la clase original.

Por ejemplo, una hamburguesa puede tener queso extra, tamaño grande y descuento aplicado, todo ello envolviendo el producto base con distintos decoradores.

6.7. Facade

Problema detectado

Crear un pedido completo requiere varios pasos:

- Asignar cliente.
- Asignar restaurante.
- Asignar repartidor.
- Añadir productos.
- Asignar método de pago.
- Añadir observadores.
- Registrar el pedido en las listas correspondientes.
- Realizar el pago.
- Notificar a los observadores.

Si todos estos pasos estuvieran repartidos directamente por el menú o por la clase de prueba, el código sería más repetitivo y más difícil de entender.

Solución aplicada

Se ha aplicado el patrón Facade mediante la clase `OrderFacade`. Esta clase ofrece un método principal, `createOrder`, que concentra todos los pasos necesarios para crear un pedido correctamente.

Desde fuera, el menú solo necesita crear una instancia de `OrderFacade` y llamar a `createOrder`, pasando los datos necesarios.

Clases participantes

- `OrderFacade`: fachada.
- `Order`: pedido que se configura.
- `Client`: cliente del pedido.
- `Restaurant`: restaurante del pedido.

- `DeliveryDriver`: repartidor asignado.
- `ServerManager`: gestor central.
- `PaymentMethodStrategy`: método de pago.
- `MenuItemComponent`: productos del pedido.
- Observadores del pedido.

Justificación

Este patrón simplifica el uso del sistema desde el menú. En vez de repetir todos los pasos de creación de un pedido, se encapsulan dentro de una fachada.

Esto hace que el código de `OptionsMenu` sea más claro y que la creación de pedidos esté centralizada en una sola clase.

7. Funcionamiento principal del programa

El programa principal se ejecuta desde la clase `Test`. Al iniciar, se crea una instancia del menú y se registran algunos datos iniciales para poder probar la aplicación más fácilmente.

El menú permite seleccionar distintas opciones:

1. Register user
2. Delete user
3. Simulate new order
4. Update order state
5. Cancel order
6. Show active orders
7. Show finished/cancelled orders
8. Delete finished/cancelled orders
9. Exit

La opción más importante es la de simular un nuevo pedido. En este flujo, el usuario debe:

- Elegir un cliente.
- Elegir un restaurante.
- Elegir productos del menú.
- Personalizar los productos si quiere.
- Elegir método de pago.
- Crear el pedido.
- Ver el pedido registrado en el sistema.

Una vez creado, el pedido puede avanzar de estado mediante la opción de actualizar pedido. Cada cambio de estado genera notificaciones para los observadores registrados.

8. Flujo de creación de un pedido

El flujo general de creación de un pedido es el siguiente:

1. El usuario selecciona la opción de simular nuevo pedido.
2. El sistema comprueba que existan clientes, restaurantes y repartidores registrados.
3. Se selecciona un cliente.
4. Se selecciona un restaurante.
5. Se muestran los productos del menú.
6. Se añaden productos al pedido.
7. Cada producto puede personalizarse con decoradores.
8. Se selecciona el método de pago.
9. `OrderFacade` se encarga de crear el pedido completo.
10. `ServerManager` asigna un repartidor.
11. Se añaden los observadores del cliente, restaurante y repartidor.
12. Se registra el pedido en las listas correspondientes.
13. Se calcula el total.
14. Se realiza el pago.

15. Se notifican los cambios.

Este flujo permite demostrar la combinación de varios patrones trabajando juntos en un mismo caso de uso.

9. Pruebas realizadas

Para comprobar el funcionamiento del sistema se han utilizado una clase principal:

- **Test**

9.1. Prueba manual con Test

La clase **Test** permite ejecutar el programa de forma interactiva por consola. Desde esta clase se puede navegar por el menú y probar manualmente el registro de usuarios, la creación de pedidos, los cambios de estado y las cancelaciones.

Esta prueba resulta útil para comprobar el comportamiento real del programa desde el punto de vista del usuario.

10. Resultados obtenidos por consola

Durante la ejecución del programa, el sistema muestra mensajes por consola para informar de lo que ocurre en cada momento.

Por ejemplo, al pagar un pedido con tarjeta se muestra un mensaje indicando el importe y el método de pago utilizado. En el caso de la tarjeta, se ocultan parcialmente los números por seguridad, mostrando únicamente los últimos dígitos.

Cuando un pedido cambia de estado, se muestran mensajes como:

```
Pedido pendiente de confirmacion
El restaurante esta preparando el pedido
El repartidor esta en camino
El repartidor ha entregado su pedido
```

También se muestran notificaciones para los observadores, por ejemplo:

Client: Carlos tu pedido #1 ha cambiado a En preparación

Restaurante: BurgerKing el pedido #1 del cliente Carlos se ha actualizado a: En preparación

Delivery driver: William el pedido #1 del cliente Carlos ha cambiado a: En preparación

Estos mensajes permiten comprobar que el patrón Observer funciona correctamente y que los cambios de estado se están notificando a las partes implicadas.

11. Reparto del trabajo

El trabajo se ha dividido entre los distintos integrantes del grupo, intentando que cada persona se encargara de una parte concreta del sistema o de un patrón de diseño.

Según la planificación del proyecto, el reparto fue aproximadamente el siguiente:

- Jose: implementación y estructura del patrón Factory Method.
- Rodrigo: implementación de la fachada para la creación de pedidos.
- Jacob y David: implementación y revisión del patrón Decorator.
- Jacob y Daniel: implementación del patrón Observer.
- Daniel: implementación del patrón Strategy.
- Daniel: implementación del patrón State.
- Jose y Rodrigo: Implementación e integración del contexto server manager
- Grupo completo: revisión del menú principal, pruebas, integración con y corrección de errores.

Aunque cada integrante trabajó en una parte concreta, la integración final necesitó revisar que todos los patrones funcionasen correctamente juntos. Esto fue especialmente importante porque varias clases se conectan entre sí a través de `Order`, `ServerManager` y `OptionsMenu`.

12. Problemas encontrados durante el desarrollo

Durante el desarrollo aparecieron algunos problemas propios de integrar varios patrones en un mismo proyecto.

Uno de los problemas fue conseguir que el patrón Observer tuviera una utilidad real y no se limitase únicamente a imprimir mensajes por consola. Para solucionarlo, se hizo que los observadores también modificaran las listas de pedidos cuando correspondiera. Por ejemplo, cuando un pedido se entrega o se cancela, el repartidor elimina ese pedido de su lista de pedidos asignados.

Otro problema fue la creación completa de pedidos. Al principio, la lógica estaba repartida en varios sitios, por lo que se decidió usar una fachada para centralizar el proceso. De esta forma, la creación de pedidos quedó más ordenada.

También hubo que adaptar los productos del menú al patrón Decorator, ya que los productos podían ser básicos o estar personalizados. Para ello se usó la interfaz `MenuItemComponent`, permitiendo que tanto los productos base como los decoradores se pudieran tratar de la misma manera.

Finalmente, se añadieron opciones al menú para mostrar pedidos activos, pedidos finalizados o cancelados, y eliminar pedidos terminados del servidor. Esto ayudó a que la simulación fuese más completa y fácil de probar.

13. Posibles mejoras futuras

Aunque el proyecto cumple con el objetivo de aplicar patrones de diseño, se podrían añadir varias mejoras en el futuro.

Una mejora importante sería implementar persistencia en una base de datos real, para que los datos no se pierdan al cerrar el programa. Actualmente, todo se guarda en listas en memoria dentro de `ServerManager`.

También se podría mejorar la validación de datos. Por ejemplo, se podrían validar mejor los nombres de usuario, los métodos de pago, los índices introducidos por consola y los importes de descuentos o extras.

Otra mejora sería añadir una interfaz gráfica o una interfaz web sencilla para que la aplicación fuese más fácil de utilizar. Actualmente funciona por consola, lo cual es suficiente para demostrar los patrones, pero no sería lo más cómodo para un usuario final.

También se podría ampliar el sistema de pedidos añadiendo:

- Direcciones.

- Tiempos estimados de entrega.
- Valoración de restaurantes.
- Historial de pedidos.
- Promociones.
- Control de disponibilidad de repartidores.

Por último, se podría mejorar el sistema de estados para permitir reglas más complejas, como impedir cancelar un pedido cuando ya está en reparto, o permitir que el restaurante rechace un pedido antes de empezar a prepararlo.

14. Conclusiones

El proyecto nos ha permitido aplicar varios patrones de diseño en un caso de uso parecido a una aplicación real. En lugar de implementar patrones de forma aislada, se han combinado dentro de un mismo sistema, lo que ayuda a entender mejor cuándo tiene sentido utilizar cada uno.

El patrón Factory Method se ha usado para crear distintos tipos de usuarios sin acoplar el código a clases concretas. Singleton se ha aplicado en `ServerManager` para tener un único gestor central de la aplicación. State ha permitido representar de forma clara el ciclo de vida de un pedido. Observer ha servido para notificar los cambios de estado a cliente, restaurante y repartidor. Strategy ha permitido seleccionar distintos métodos de pago. Decorator ha sido útil para personalizar productos del menú sin crear muchas clases diferentes. Finalmente, Facade ha simplificado la creación completa de un pedido.

En general, el proyecto demuestra cómo los patrones de diseño pueden ayudar a organizar mejor el código, separar responsabilidades y facilitar futuras ampliaciones.

Aunque la aplicación es una simulación por consola, la estructura permite ver claramente cómo se podría ampliar hacia un sistema más completo en el futuro.